

AXL: A Standalone Proof-Carrying Compiler for Agent Spending Bounds

Manuel Guilherme Galmanus

Bluewave AI Research

manuel@bluewaveai.online

Preprint · June 2026

Authored solely by Manuel Guilherme Galmanus. The work grew out of the author’s agent-payment infrastructure in the Stellar ecosystem (Slippay) and was developed and shared in the context of the NEAR X ecosystem.

Abstract

Autonomous agents increasingly hold and move funds. The dominant safety mechanism is a runtime check inside the agent’s host or wallet — but a runtime check is exactly the thing a compromised host can skip, and a counterparty has no way to confirm, before settling, that the check was ever run. AXL is a small declarative language and a standalone Rust compiler (`axlc`) that take a different stance: a spending policy is compiled to a logical *obligation*, discharged by an SMT solver via *k*-induction, and emitted as a tamper-evident *certificate* that any third party can re-verify offline against the exact policy bytes. The compiler has zero crate dependencies (standard library only; the solver is invoked as a subprocess and its absence fails closed). For the deployed sliding-window policy we prove a tight, minimal bound: total outflow in *any* real-time window of length W is at most $2 \times \text{window_cap}$ — the factor of two being a provable consequence of epoch-boundary straddling, not a safety margin. We describe the language (four orthogonal directives), the SMT lowering, the certificate format and its SHA-256 binding, and a browser-side re-verifier that regenerates the same obligations without a solver. We are deliberate about scope. AXL today is an *offline verifier*, not a deployed control: the on-chain contract enforces the same $2\times$ bound by hard-coded logic and treats the policy hash as provenance only; no continuous-integration gate yet couples proof to deployment. We state these gaps as falsifiable conditions, because the contribution is the verifiable artifact and its construction, not a claim that it is currently load-bearing in production.

Keywords: proof-carrying code, SMT, *k*-induction, agent autonomy, spending bounds, formal verification, smart wallets, AP2/x402.

1 Introduction

An autonomous agent that can pay needs a bound it cannot exceed. The standard way to impose one is a runtime guard: the wallet or the orchestration host checks, on each transaction, that the agent is within budget. This works until the host is the adversary. A prompt-injected

or compromised agent runtime can route around its own guard; a counterparty receiving payment has no way to inspect the payer’s internal control and confirm, *before* accepting settlement, that any bound is in force. The check is real but unverifiable from the outside, and unverifiable-from-the-outside is the wrong property for money moving between mutually distrusting parties.

The classical answer to “trust the producer’s claim about code it ran” is proof-carrying code (Necula, 1997): the producer ships a machine-checkable proof that its artifact satisfies a policy, and the consumer checks the proof rather than trusting the producer. AXL applies that stance to agent spending. A policy is written in a tiny declarative language; the compiler reduces the quantitative part of the policy to a logical obligation; an SMT solver (de Moura & Bjørner, 2008) discharges it by k -induction (Sheeran et al., 2000); and the result is a small JSON *certificate* cryptographically bound to the exact policy text. Anyone — a counterparty, an auditor, an insurer — can re-run the verification offline and either confirm the bound or detect that the certificate has drifted from the policy.

What this paper claims, and what it does not. Honest positioning is load-bearing here. Agent *payment* is not novel: x402 carries machine payments over HTTP 402 and Google’s AP2 standardizes human-approved spending mandates. AXL is orthogonal to both — it does not move money; it proves a property of the policy that governs an agent that does. The contribution is (i) a language whose quantitative directive lowers cleanly to a decidable SMT obligation, (ii) a zero-dependency compiler that produces a re-verifiable certificate with a tight, machine-checked bound, and (iii) a faithful browser re-verifier. The contribution is *not* a deployed control: §8 states plainly that the on-chain contract enforces the bound by hard-coded arithmetic and consumes no certificate, that no CI gate yet couples proof to deployment, and that the underlying SMT model is small enough (≈ 40 lines) that the proof itself is not a barrier to entry. We treat these as falsifiable gaps to close, not as features to obscure.

Contributions.

- **AXL, a four-directive policy language** (§2) separating capability scope (bind), output schema (constrain), decidable predicates (prove), and the one SMT-discharged quantitative invariant (invariant). All four are enforced by code or solver, never by a language model.
- **A standalone compiler** (§3) with no crate dependencies: hand-rolled lexer, parser, JSON, and SHA-256, lowering the sliding-window invariant to SMT-LIB and discharging it via a subprocess solver, fail-closed when no solver is present.
- **A tight, minimal, machine-checked bound** (§4): for `sliding_window(ceiling = 2)` the real-time-window outflow bound $K = 2$ is proved inductive, attainable (tight), and minimal ($K = 1$ is not inductive). The deployed wallet enforces exactly this constant.
- **A proof-carrying certificate** (§5) bound by SHA-256 to the policy bytes, with a defined

load-bearing projection and drift detection, plus a browser re-verifier that regenerates the identical obligations without a solver (§6).

- **An explicit scope-and-limits section** (§8) with falsifiable statements about what is and is not in the production path.

2 The AXL Language

An AXL program is an agent block with up to four orthogonal directives. Each addresses a different layer of an agent’s action, and each is default-deny / fail-closed: an unrecognized construct is a compile error, never a silent pass.

`bind -> [tools]` declares the capability scope. A capability that is not bound has no schema in the action space, so a well-formed tool call for it cannot be emitted. An unknown capability raises `UnknownCapability` rather than being dropped silently.

`constrain -> Schema` fixes the output schema, compiled to constrained decoding (`json_schema` with strict mode). A strict schema must declare `additionalProperties: false`, or the compiler refuses with `StrictSchemaNeedsAdditionalPropertiesFalse`.

`prove -> <predicate>` declares a decidable predicate over the output, of the form *lhs op rhs* with $op \in \{<, \leq, >, \geq, =, \neq, \text{in}\}$. Predicates are compiled to deterministic checks and evaluated at enforcement time; output violating any predicate is rejected.

`invariant -> sliding_window(ceiling = M) bound K` is the only directive sent to the SMT solver. It declares a sliding-window spending policy with aggregate ceiling $M \cdot \text{window_cap}$ and a *claimed* real-time-window bound $K \cdot \text{window_cap}$. Any other policy family (for example `token_bucket`) is rejected with `UnsupportedPolicy` *before* any solver runs.

The separation is deliberate: `bind` and `constrain` shape what the agent *can* express, `prove` checks decidable facts about a specific output, and `invariant` is the one place where a *quantitative safety property over all possible action sequences* is discharged by proof rather than by per-call checking.

2.1 Worked examples

The repository ships a matrix of examples exercising every branch of the decision procedure. The deployed agent policy is the strict, sub-unity case $M = 2$:

```
agent AgentWallet {
  bind      -> [read_balance, propose_payment]
  invariant -> sliding_window(ceiling = 2) bound 2
}
```

A non-invariant policy uses decidable predicates only:

Example	Policy	Verdict	Exit
agent_wallet_m1	sliding_window(ceiling=1) bound 1	ISSUED, $K=1$, tight	0
agent_wallet_m2	sliding_window(ceiling=2) bound 2	ISSUED, $K=2$, tight	0
agent_wallet_m3	sliding_window(ceiling=3) bound 2	ISSUED, $K=2$, tight + diagnostic	0
agent_wallet_none	sliding_window(ceiling=none) bound 2	REFUSED (unbounded)	2
agent_wallet_unsupported	token_bucket(ceiling=2) bound 2	REFUSED (unsupported)	1

Table 1: The example matrix. The $M=3$ case is instructive: the weighted throughput check binds tighter than the nominal ceiling, so the prover issues $K=2$ and attaches a diagnostic that the declared ceiling is looser than the proved bound.

```

agent Settlement {
  bind      -> [read_balance, propose_settlement]
  constrain -> SettlementDecision
  prove     -> decision.amount <= account.balance
  prove     -> decision.recipient in account.allowlist
}

```

Table 1 summarizes the full matrix and the verdict each yields. The two failing cases are first-class: an unbounded policy (`ceiling = none`) and an unsupported family (`token_bucket`) both *refuse*, with distinct exit codes, so a missing or unsupportable proof can never be mistaken for a passing one.

3 The Compiler

`axlc` is a standalone Rust crate (`axl-compiler 0.1.0`) with *no* [dependencies] section — JSON, parsing, and SHA-256 are implemented against the standard library only. The SMT solver is the single external requirement, and it is a *runtime* dependency invoked as a subprocess, not a linked crate; its absence makes the prover fail closed rather than silently pass. This keeps the trusted computing base of the verifier small and auditable: a reader can read the entire compiler without chasing transitive dependencies.

3.1 Pipeline

1. **Parse** (`parse.rs`): `parse_agent` produces an `AgentSpec` with capabilities, schema, predicates, and an optional `InvariantDecl` (family, ceiling, bound), where the ceiling is a typed `Multiplier(u64)` or `None`.
2. **Compile** (`compile.rs`): `compile_agent` resolves the toolset and schema into an `InferenceContract` and emits the request configuration (constrained decoding).
3. **Predicates** (`predicate.rs`): `compile_predicate` parses each `prove` directive into a `Predicate` (lhs, op, rhs) evaluated with JavaScript-faithful comparison semantics.

4. **SMT emission** (`smt.rs`): for an invariant, emit lowers each proof obligation to SMT-LIB text.
5. **Discharge** (`prove.rs`): `detect_backend` probes for a solver, `check` runs it, and `discharge` returns a `Certificate` (`Issued` / `Refused`).
6. **Certify** (`certify.rs`): `build_certificate` assembles the JSON artifact and binds it by SHA-256 to the policy bytes; `verify` re-derives and compares.

3.2 Solver dispatch, fail-closed

`detect_backend` probes, in order, a `z3` binary on `PATH` (fed SMT-LIB on `stdin`) and a `python3` interpreter with the `z3-solver` package. Verdict parsing reads the *last* `sat`/`unsat`/`unknown` token from solver output, so solver warnings cannot corrupt the result. If no backend is found, `discharge` returns `NoSolver` and the caller refuses: the absence of a checker is never a pass.

4 The Sliding-Window Model and the Safety Property

The quantitative core of AXL is a single safety invariant over a two-epoch abstraction of a sliding-window rate limiter, discharged by k -induction.

4.1 State and transition

Let W be the window length (seconds) and let `cap` abbreviate `window_cap`. The state is a pair (p, c) giving outflow in the previous and current epoch. A charge of amount a arriving elapsed seconds into the current epoch transitions the state as follows (`smt.rs`), writing $rolled \equiv (\text{elapsed} \geq W)$:

$$\begin{aligned}
 p_1 &= rolled ? (\text{elapsed} < 2W ? c : 0) : p && \text{(carry / drop / keep)} \\
 c_1 &= rolled ? 0 : c \\
 eie &= rolled ? 0 : \text{elapsed} && \text{(elapsed-in-epoch)} \\
 weighted_prev &= \lfloor p_1 \cdot (W - eie) / W \rfloor && \text{(throughput shaping)} \\
 accept &\equiv (weighted_prev + c_1 + a \leq \text{cap}) \wedge (p_1 + c_1 + a \leq M \cdot \text{cap}) \\
 c_2 &= c_1 + a.
 \end{aligned}$$

The acceptance test combines a *weighted throughput* check (which smooths spending across the boundary) with a hard *aggregate ceiling* $M \cdot \text{cap}$.

4.2 The invariant and the obligations

Definition 1 (Window invariant). For a claimed bound K ,

$$inv_K(p, c) \equiv 0 \leq p \leq \text{cap} \wedge 0 \leq c \leq \text{cap} \wedge p + c \leq K \cdot \text{cap}.$$

The compiler emits up to four obligations and checks each verdict against its expected value; any deviation refuses:

- **Base:** $\neg \text{inv}_K(0,0)$ is unsat (the initial state satisfies the invariant).
- **Inductive:** $\text{inv}_K(p,c) \wedge \text{accept} \wedge \neg \text{inv}_K(p_1,c_2)$ is unsat (every accepted charge preserves the invariant).
- **Attainable:** $\text{inv}_K(p,c) \wedge \text{accept} \wedge (p_1 + c_2 = K \cdot \text{cap})$ is sat (the bound is reachable, hence tight rather than loose).
- **Unbounded** (only for `ceiling = none`): a witness $n \cdot \text{per_tx} > 1000 \cdot \text{cap}$ is sat, forcing refusal.

Theorem 1 (Deployed bound). *For the deployed policy `sliding_window(ceiling = 2)`, the invariant inv_2 is inductive under `accept`: every accepted charge from a state satisfying inv_2 lands in a state satisfying inv_2 . Consequently the total outflow in any real-time window of length W is at most $2 \cdot \text{window_cap}$. The bound $K = 2$ is tight (attainable) and minimal (inv_1 is not inductive).*

Proof (machine-discharged). The Base and Inductive obligations for $K = 2$ are emitted as SMT-LIB and discharged unsat by the solver; unsat on the negated inductive step is exactly the statement that no accepted transition can break inv_2 . The Attainable obligation is sat, exhibiting a charge sequence reaching $p + c = 2 \cdot \text{cap}$, so the bound is not loose. The corresponding inductive obligation for $K = 1$ is *not* unsat, so 1 is not a sound bound; hence 2 is minimal. The real-window consequence follows because two epochs of length W overlap any real W -length interval at most pairwise, so accumulated outflow over such an interval is bounded by $p + c$. $\square$

Remark (The factor of two is structural, not slack). The $2\times$ is a provable consequence of *epoch-boundary straddling*: an adversary spends up to `cap` immediately before a boundary and up to `cap` immediately after, and both charges fall inside one real W -length interval. The weighted-throughput term shapes *when* spending lands but cannot defeat the straddle; $2 \cdot \text{cap}$ is therefore the honest worst case for a single-window abstraction, and AXL proves it rather than assuming a tighter number that does not hold. Disanalogy with a fixed calendar-window limiter: there the per-window cap is exactly `cap` by construction, but it permits a $2\times$ burst across the reset instant *without* reporting it; AXL’s sliding model surfaces the same $2\times$ explicitly and certifies it.

5 The Proof-Carrying Certificate

`axlc certify` emits a deterministic JSON certificate that binds the verdict to the exact policy bytes. An ISSUED certificate for the deployed policy has the shape:

```
{
  "kind": "axl-proof-certificate",
  "axl_version": "0.1.0",
```

```

"spec_sha256": "<sha256 of the policy bytes>",
"agent": "AgentWallet",
"invariant": { "family": "sliding_window", "ceiling": 2, "bound": 2 },
"verdict": "ISSUED",
"tight": true,
"diagnostic": null,
"onchain": {
  "ssl_hash": "<= spec_sha256>",
  "window_cap_multiplier": 2,
  "claim": "real-time window outflow <= 2 * window_cap",
  "matches_deployed_enforcement": true
},
"backend": "z3 binary (PATH)"
}

```

The binding is a hand-rolled FIPS 180-4 SHA-256 of the policy bytes: editing the policy invalidates the certificate. A *load-bearing projection* — kind, axl_version, spec_sha256, agent, family, ceiling, bound, verdict, tight, onchain.ssl_hash, and onchain.window_cap_multiplier — defines exactly which fields are compared on re-verification; backend and the free-text claim are metadata. A REFUSED certificate carries verdict = "REFUSED", a refused_reason, and a null onchain block, so a refusal is a first-class, signable outcome rather than an absent file.

What is, and is not, carried. The certificate carries the proved bound K , the policy SHA-256, and the verdict — not the SMT-LIB text and not the solver’s internal proof object. “Proof-carrying” here means the bound claim is *independently re-derivable*: verify-cert re-emits the obligations from the certificate’s (*family, ceiling, bound*), re-runs the solver, and asserts the verdict and projection match. Drift between certificate and policy is caught (exit 2); a certificate minted for one policy does not verify against another, because the SHA-256 differs.

Disanalogy with classical proof-carrying code (Necula, 1997): there the proof object itself travels and the consumer checks it directly with a fixed proof checker; here the *claim* travels and the consumer re-discharges the obligation with its own solver. The trust shifts from “check this proof” to “re-run this small, fully specified verification” — weaker if the consumer lacks a solver, but the obligations are tiny and the policy binding is cryptographic.

6 Verification

6.1 Offline browser re-verification

axlVerify.ts re-verifies a certificate client-side with no solver dependency. It (i) re-hashes the policy via WebCrypto and asserts it matches spec_sha256 (cryptographic tamper-evidence); (ii) checks structural coherence (kind, verdict, ssl_hash = spec_sha256, window_cap_multiplier = K); and (iii) *regenerates* the exact SMT-LIB obligations axlc discharged, via a TypeScript port of the emitter whose preamble, transition definitions, and invariant expressions are byte-for-byte equivalent to smt.rs. The browser thus confirms

the binding and reproduces the theorems without running a solver; discharging them automatically in-browser (e.g. via a WASM solver) is identified as future work.

6.2 Relationship to on-chain enforcement

We state this precisely to avoid overclaiming. The Soroban smart-wallet contract enforces the spending policy by *hard-coded* arithmetic — a weighted throughput check and a hard ceiling $prev_spent + cur_spent + amount \leq 2 \cdot window_cap$ — and stores the policy hash (`ssl_hash`) as immutable *provenance only*. The contract does not read or verify any certificate. The relationship between AXL and the contract is therefore *confirmatory*: AXL independently re-derives and machine-checks the same $2\times$ constant the contract already encodes. AXL does not, today, feed the bound to the contract, and no spend is cryptographically conditioned on the proved policy.

7 Evaluation

The compiler is exercised by integration tests (`invariant_prove.rs`, `certify.rs`) covering: parse-level rejection of malformed and unsupported invariants; structural assertions on emitted SMT-LIB; end-to-end solver discharge (skipped when no solver is installed); CLI exit codes; and certificate round-trip, drift detection, and spec-swap rejection. The asserted verdicts are exactly those of Table 1: $M=2 \rightarrow K=2$ tight, $M=1 \rightarrow K=1$ tight, $M=3 \rightarrow K=2$ tight with a looser-ceiling diagnostic, `ceiling=none` $\rightarrow$ REFUSED, and an overclaimed bound $\rightarrow$ REFUSED. The library is written to return `Result` on every parse and JSON error and never to panic on malformed input. Concrete size: the SMT model is on the order of 40 lines of SMT-LIB; the compiler has 0 external crate dependencies; the certificate’s load-bearing projection is 11 fields.

8 Scope and Limitations

AXL applies only to *decidable* policy: arithmetic bounds, membership, and bounded state. Irreducibly fuzzy judgments (“is this transaction suspicious?”) are out of scope and still require a model in the loop. Beyond that boundary, we name the gaps between the artifact and a deployed control, each as a falsifiable statement.

- **No CI gate couples proof to deployment.** There is no workflow that runs `verify-cert` on change. *Falsifiable*: until a required check rejects a merge whose policy and certificate disagree, the proof can silently drift from the deployed contract on any commit.
- **The on-chain constant is hard-coded, not certificate-driven.** The contract enforces $2\times$ as a literal. *Falsifiable*: if AXL proved $K = 3$ for a revised policy, the contract would not track it without a manual edit; proof and enforcement are not yet a single source of truth.
- **The policy hash is provenance, not a gate.** `ssl_hash` is pinned immutably but never interpreted on-chain. *Falsifiable*: no spend is rejected on the basis of a hash mismatch with a certified policy.

- **The proof is not a moat.** The SMT model is ≈ 40 lines; a competent formal-methods engineer reproduces it in engineer-weeks. The only durable advantage would be *adoption* of the certificate as a required-to-participate artifact (a second prover, an insurer, an on-chain registry), which does not exist today.
- **The solver is a runtime dependency.** `prove`, `certify`, and `verify-cert` require `z3` (binary or Python package). This is intentional and fail-closed — absence refuses — but it means a consumer without a solver can check only the binding and the obligation text, not the discharge.

We surface these because the value of a proof-carrying scheme is exactly its honesty about what has and has not been checked. The certificate machinery — language, lowering, k -induction discharge, SHA-256 binding, deterministic re-verification, drift detection — is real and tested. The forcing function that would make it load-bearing in production is the work that remains.

9 Related Work

Proof-carrying code (Necula, 1997) originates the producer-ships-proof, consumer-checks-proof stance; AXL adapts it to a re-discharge model bound by a content hash (§5). **SMT solving** (de Moura & Bjørner, 2008) and **k -induction** (Sheeran et al., 2000) are the engine and the proof technique for the sliding-window invariant. **Smart-contract verification** (e.g. specification and model checking of on-chain logic) targets the contract itself; AXL instead certifies the *off-chain policy* an agent commits to, and (today) only confirms the contract’s hard-coded bound rather than generating it. **Agent-payment standards** — x402 for HTTP-native machine payments and Google’s AP2 for human-approved mandates — provide the rails and the authorization chain; AXL is orthogonal, proving a quantitative property of the governing policy rather than transporting value or capturing consent. **Capability security** informs the `bind` directive’s default-deny action-space scoping.

10 Conclusion

AXL reframes an agent’s spending limit from a runtime check that a compromised host can skip into a logical obligation that anyone can re-verify offline. The compiler is small and dependency-free by construction, the sliding-window bound is proved tight and minimal by k -induction, and the certificate is bound to its policy by a content hash so that drift is detectable. We have been explicit that the scheme is, today, an offline verifier rather than a deployed control: the on-chain contract enforces the same bound by hard-coded arithmetic and consumes no certificate, and no CI gate yet couples proof to deployment. Those are the falsifiable conditions whose resolution would turn a verifiable artifact into a load-bearing one. The thesis is narrow and, we think, correct: for money moving between agents that do not trust each other, a bound the counterparty can re-check beats a bound it must take on faith.

References

- Necula, G. C. (1997). Proof-carrying code. In *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 106–119.
- de Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, 337–340.
- Sheeran, M., Singh, S., & Stålmarck, G. (2000). Checking safety properties using induction and a SAT-solver. In *Formal Methods in Computer-Aided Design (FMCAD)*, LNCS 1954, 127–144.

Correspondence: Manuel G. Galmanus, Bluewave AI Research. This is a preprint and has not been peer reviewed. All quantitative claims about the compiler (verdicts, exit codes, bound, dependency count) are reproducible from the open-source `axl-compiler` crate; the bound in Theorem 1 is machine-discharged by an SMT solver and re-verifiable with `axlc verify-cert`. The limitations in Section 8 are stated as falsifiable conditions.